Trabalho Prático 1 - BBC Search

João Carlos Ferraz de Sousa - 2023087990
Luiz Felipe Gondim Araujo - 2023028188

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte - MG - Brasil

1 Introdução

Esta documentação busca relatar com certa profundidade o trabalho criativo que foi feito pelo nosso grupo na elaboração do Trabalho Prático 1 de Algoritmos 2. Nesse trabalho, o objetivo era exercitar o conhecimento adquirido em sala acerca de tries e ao mesmo tempo ter a oportunidade de abordar conceitos não vistos na disciplina, no caso específico, bibliotecas para construção de aplicações web. Com isso, foi possível implementar uma aplicação que seja capaz de realizar buscas em um corpus de notícias da BBC News.

2 Implementação

O programa foi desenvolvido em Python, HTML5 e CSS. Em relação ao ambiente de desenvolvimento, consistiu na criação de um ambiente virtural python "env", e sua escolha visou isolar as dependências do projeto, evitando possíveis conflitos nos diferentes computadores em que a aplicação foi feita.

Para mais, a aplicação foi dividida em dois módulos principais. O primeiro, denominado Core, foi criado com o intuito de conter toda a lógica de processamento de dados e consulta do sistema. Enquanto o módulo app, foi construído para lidar com o Flask, a parte que possui um contato maior com o usuário final. Nas seções a seguir, haverá explicações acerca de cada módulo.

3 Core

O módulo em questão trabalha com os arquivos de notícias da empressa BBC News disponibilizados por Greene e Cunningham (2006). Com um total de 2225 arquivos, foi pensado em organizá-los com o formato `num_categoria`.txt na pasta **data**, sendo "num"o nome inicial do arquivo no corpus e "categoria"o nome da pasta onde ele estava (008_sport.txt, por exemplo). Dessa maneira, cada notícia possui um identificador único, facilitando nossas buscas ao decorrer da aplicação.

3.1 Trie compacta

Urge relatar que no começo do desenvolvimento ambas as partes envolvidas sentiram dificuldade em entender o porque do uso da trie no trabalho. "Não seria mais fácil usar um dicionario python enorme para representar o indice?"ou "porque eu usaria a trie, ela parece ocupar muito espaço"foram alguns dos questionamentos iniciais propostos. A questão é que para indexar muitas palavras com

vocabulário rico (que é o nosso caso) a trie compacta acaba escalando muito melhor, o uso de mémoria acaba sendo menor porque existem muitos prefixos compartilhados e é mais tranquila para armazenar informações adicionais, em nó, por exemplo. Portanto, para implementar essa estrutura de dados, foram utilizados os métodos a seguir:

3.1.1 `insert_word`

Esse método é chamado pelo indexer e recebe uma palavra como parâmetro, sua funcionalidade se baseia em adicionar essa string na trie compacta. Para tanto, ela se depara com três possíveis cenários: já existe um nó contendo exatamente essa palavra, existe um nó contendo apenas um prefixo da palavra e não existe prefixo comum entre a palavra e outros nós. O método é capaz de identificar em cada iteração qual o cénario que está lidando, remover/adicionar nós e **incrementar** a quantidade de vezes que uma determinada palavra apareceu em um documento específico.

3.1.2 `search_word`

Método usado pelo retriever para buscar palavras dentro da trie, ele entra em looping e busca constantemente prefixos em comum entre a palavra pesquisada e os filhos de um nó até encontrar um casamento. Ele retorna os documentos e o número de aparições daquela determinada chave.

3.1.3 `save_to_disk`

Método résponsavel por armazenar a trie em disco, evitando assim o custo de criar ela no início de toda execução. O formato escolhido foi JSON por ser mais simples de implementar, basta usar uma função auxiliar para converter os nós num tipo de dicionário em série. Ela cria o arquivo no diretório escolhido caso ele não exista.

3.1.4 `load_from_disk`

Procura no diretório o arquivo JSON contendo a trie e reconstroi ela na memória principal usando uma função auxiliar de leitura que converte o dicionário de volta em nós.

3.2 Indexer

Além de gerenciar a trie compacta, essa estrutura de dados armazena o nome dos documentos lidos e serve de ponte entre o retriever e a trie, possuindo assim um papel fundamental nesse projeto. Ao ser inicializado ele busca um arquivo JSON armazenado em disco para carregar uma trie, caso não exista, ele possui o método `add_file` para adicionar todas as palavras de um documento nela.

3.3 Retriever

A classe `Retriever` foi desenvolvida para constituir a parte principal da recuperação de informações do sistema de busca. Sua função é interpretar consultas textuais fornecidas pelo usuário, processá-las de acordo com operadores booleanos e realizar a busca correspondente sobre o índice invertido previamente construído. Para isso, foram implementadas os seguintes métodos:

3.3.1 `_parse`

A função `_parse` é responsável por construir a árvore de análise (*parse tree*) que representa a estrutura lógica da consulta booleana informada pelo usuário. A construção dessa árvore segue o

algoritmo *Shunting-Yard*, proposto por Edsger Dijkstra, que converte expressões infixas em uma forma hierárquica apropriada para avaliação recursiva.

Durante o processo, os termos simples (palavras) são encapsulados em nós do tipo `Node`, enquanto os operadores lógicos `AND` e `OR` são tratados com diferentes precedências. Sabendo disso, o operador `AND` possui precedência maior que `OR`, assegurando que conjunções sejam avaliadas antes de disjunções e os parênteses são respeitados para impor agrupamentos explícitos. Ao término da execução, a função retorna a raiz da árvore sintática abstrata (AST) correspondente à consulta.

3.3.2 `_evaluate`

A função `_evaluate` realiza a avaliação recursiva da árvore de análise construída por `parse`. Cada nó da árvore pode representar um termo ou um operador lógico (`AND`, `OR`). Quando o nó é um termo simples, a função consulta o índice invertido e retorna o dicionário de documentos em que o termo aparece, juntamente com suas frequências de ocorrência.

Nos casos em que o nó representa um operador lógico, a função aplica as operações correspondentes:

- Para o operador `AND`, realiza-se a interseção dos conjuntos de documentos retornados pelos subnós esquerdo e direito, somando as frequências dos termos que ocorrem em ambos os documentos;
- Para o operador `OR`, calcula-se a união dos conjuntos de documentos, combinando as frequências de modo aditivo.

Desse modo, a função `_evaluate` percorre toda a árvore de forma descendente e retorna um dicionário contendo, para cada documento, a soma ponderada das frequências dos termos compatíveis com a consulta lógica especificada.

3.3.3 `_rank_results`

No módulo de recuperação adotou-se uma estratégia de *scoring* baseada em *z-score* calculado por termo, a fim de evitar comparações indevidas entre distribuições de frequência heterogêneas. Para cada termo t da consulta, extraem-se do índice as frequências $f_{t,d}$ nos documentos d em que t ocorre. Sejam N_t o número de documentos que contêm o termo t. Calculam-se então a média μ_t e o desvio-padrão σ_t por termo:

$$\mu_t \;=\; \frac{1}{N_t}\sum_d f_{t,d}, \qquad \sigma_t \;=\; \sqrt{\frac{1}{N_t}\sum_d \big(f_{t,d}-\mu_t\big)^2}. \tag{1}$$

Para evitar divisão por zero, adota-se o *fallback* $\sigma_t = 1$ quando a variância for nula. O z-score do termo t no documento d é definido por:

$$z_{t,d} \;=\; \frac{f_{t,d}-\mu_t}{\sigma_t}. \tag{2}$$

Para cada documento d considera-se o conjunto T_d dos termos da consulta presentes nesse documento. O termo mais relevante em d é aquele que maximiza o z-score local:

$$t_d^* \;=\; \arg\max_{t\in T_d} z_{t,d}. \tag{3}$$

O score global do documento utilizado para ordenação corresponde à média aritmética dos z-scores dos termos de consulta presentes em d:

$$S_d = \frac{1}{|T_d|} \sum_{t \in T_d} z_{t,d}. \tag{4}$$

Esta formulação assegura que termos raros mas desproporcionalmente frequentes num determinado documento obtenham maior peso (ao contrário de um cálculo global que diluíria sua importância), preserva comparabilidade estatística entre termos e permite identificar explicitamente o termo que mais contribuiu para a relevância de cada resultado.

3.3.4 `_generate_snippet`

A função `_generate_snippet` tem por objetivo criar um trecho textual (*snippet*) que contextualize a ocorrência do termo mais relevante em cada documento. O procedimento inicia-se pela leitura completa do arquivo correspondente, incluindo o título. Em seguida, realiza-se uma busca pela primeira ocorrência do termo de maior z-score no texto (busca *case-insensitive*), e define-se uma janela de aproximadamente 80 caracteres antes e depois da ocorrência.

O trecho selecionado é então destacado visualmente com a inserção de uma marcação `<span class="highlight≫...</span>` em todas as instâncias do termo dentro da janela. Caso o trecho extraído não corresponda ao início ou fim do texto, são adicionadas reticências para indicar truncamento. Essa abordagem assegura que o usuário visualize o termo em seu contexto imediato, reforçando a interpretação semântica do resultado.

Além disso, a função retorna simultaneamente o título do documento, de modo que a interface possa exibir tanto o contexto textual quanto a identificação do conteúdo de origem.

3.4 Utils

Contém dois métodos úteis que vão ser utilizados em diferentes setores do código, como o `normalize` que recebe o conteúdo(string) oriundo dos arquivos, torna minúsculo, retira todos os caracteres que não são letras e números. Outra função é o `disk_exists` que é responsável pela funcionalidade, anteriormente mencionada na sessão Indexer, de verificar se existe um disco armazenando uma trie no diretório designado.

4 FrontEnd

Foi utilizado o *framework* Flask, conforme solicitado no enunciado do trabalho, para a implementação do servidor web e integração entre as camadas de aplicação e apresentação. Com isso, adotou-se uma estrutura de diretórios padronizada, dividindo o projeto em módulos de *controllers*, *templates* e arquivos estáticos, de modo a garantir separação entre a lógica de negócio e os componentes visuais. O diretório `templates/` contém os arquivos HTML, enquanto `static/` armazena os recursos estáticos, como CSS, Javascript, imagens e ícones SVG. O arquivo `index.html` é responsável por compor a interface principal do buscador, recebendo do *controller* as informações processadas pelo Core e exibindo-as ao usuário de forma organizada. Ainda há o componente `news.html` que evidencia o conteúdo de uma notícia ao clicar no seu snippet.

A interface do buscador foi pensada para ser simples, funcional e adaptável a diferentes tamanhos de tela, sendo estruturada no arquivo `index.html`. Logo no topo da página, encontra-

se um campo de busca centralizado, que constitui o ponto de interação inicial do usuário. Os resultados da busca são exibidos em cartões (*cards*) que apresentam o título da notícia, um trecho relevante (*snippet*) e informações complementares, como o nome do arquivo e a categoria, esta última é representada por ícones temáticos conforme o tipo da notícia (esporte, negócios, ect). Em relação à paginação, ela foi implementada de maneira simples e intuitiva, permitindo navegar entre diferentes conjuntos de resultados por meio dos botões "Anterior" e "Posterior", além de indicar claramente o número da página atual e o total de páginas disponíveis. Além disso, cada cartão é interativo, ao clicar sobre ele, o usuário pode visualizar a notícia completa, de forma semelhante ao funcionamento de um motor de busca tradicional, como o Google. Outro ponto de interação implementado é modo claro e escuro que alteram dinamicamente o esquema de cores, garantindo boa legibilidade em diferentes condições de iluminação.

O *controller* responsável pela busca, denominado `SearchController`, foi desenvolvido um mecanismo de tratamento da consulta inserida pelo usuário. O sistema realiza uma validação da string de entrada, impedindo consultas malformadas ou com conectores lógicos posicionados incorretamente, retornando uma mensagem de erro descritiva em caso de inconsistência. Caso a consulta seja válida, o `controller` invoca o módulo `Retriever`, que executa a busca no índice invertido e retorna uma lista de documentos ranqueados conforme seu *score* de relevância. Assim, o `SearchController` atua como intermediário entre a interface e o núcleo de recuperação, assegurando que apenas consultas válidas cheguem ao motor de busca e que os resultados sejam devidamente formatados antes de serem exibidos no front-end.

5 Instruções de instalação e execução

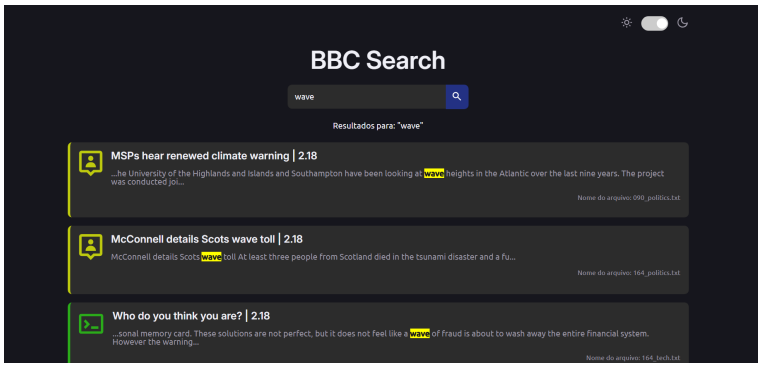
Para acessar a aplicação utilize o link manipulacao-de-sequencias.vercel.app, ou efetue os seguintes passos para instalação em seu computador (Linux):

- Instale a virtualenv no seu sistema caso nao tenha: apt install python3.10-venv;
- Crie uma virtualenv na pasta raiz: python3 -m venv env;
- Ative a env: source env/bin/activate;
- Em seguida, instale as dependências: pip install -r requirements.txt;
- Inicie a aplicação: python3 app.py;
- Acesse o link http://127.0.0.1:5000/ para ver o site;

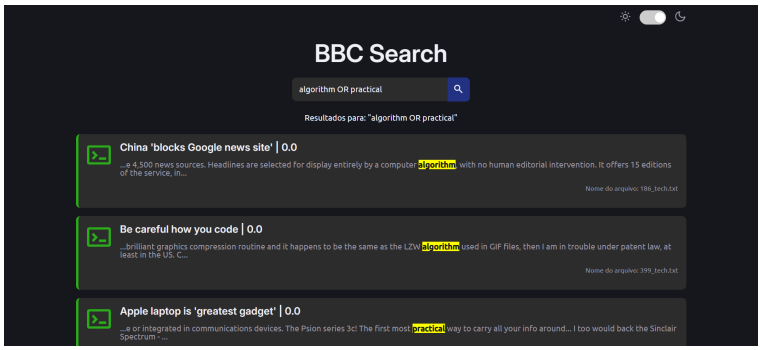
6 Exemplos de funcionamento

O mecanismo de busca da aplicação BBC Search é capaz de lidar com diversos tipos de consulta, sendo elas:

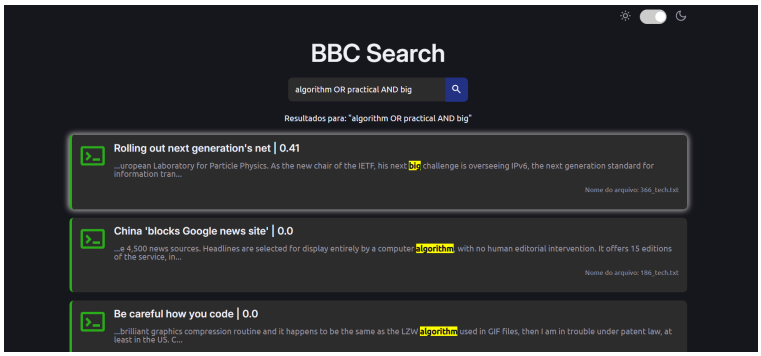
- **Única:**



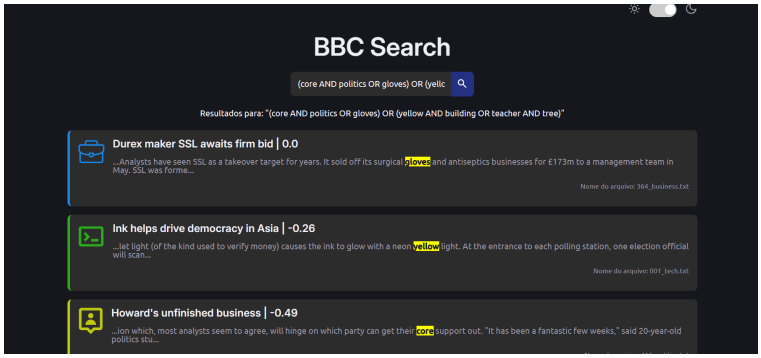
- **Com operador:**



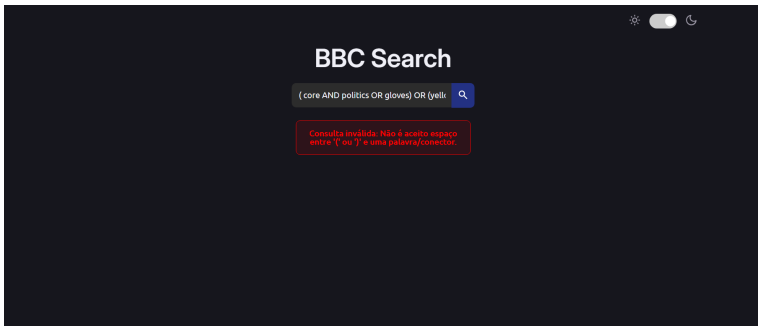
- **Multiplos operadores:**

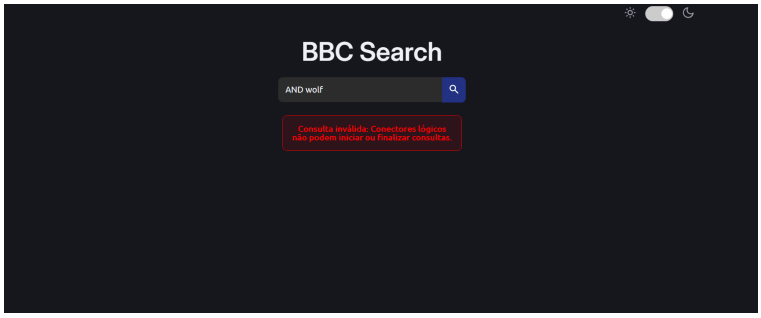


- **Complexa:**

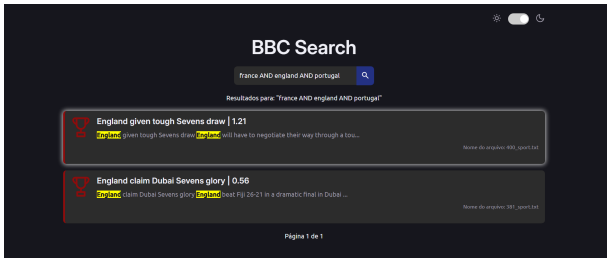


- **Cenário de erro:**





- **Consulta com clique:**



Nas imagens é possível visualizar outras funções auxiliares impletadas que enriquecem a aplicação, como ícones específicos para cada categoria de notícia, tema escuro/claro, mensagens de erro personalizadas, dentre outras previamente relatadas nessa documentação.

7 Considerações Finais

A proposta do trabalho prático foi de muitas formas gratificante a nível de desenvolvimento acâdemico, por exigir pela primeira vez no curso o uso de Front-end e por aprofundar nossos conhecimentos acerca de Tries. Além disso, nos ajudou a nível interpessoal, pois trabalhar em grupo seja pessoalmente ou através do GitHub é uma habilidade muito exigida no mercado de trabalho.

Referências

[1] TUTORIALS POINT. Compressed Tries. Disponível em: https://www.tutorialspoint.com/data_structures_algorithms/compressed_tries.htm. Acesso em: 19 out. 2025.

[2] XTECNA. Algoritmo de Shunting-Yard. Disponível em: https://xtecna.gitbook.io/solucoes-da-beecrowd/base-teorica/algoritmos-raros/algoritmo-de-shunting-yard. Acesso em: 19 out. 2025.

[3] STATISTICS HOW TO. Z-Score: Definition, Formula and Calculation. Disponível em: https://www.statisticshowto.com/probability-and-statistics/z-score/. Acesso em: 21 out. 2025.